

PRACTICAL – 01

```
document1 = "The quick brown fox jumped over the lazy dog."
document2 = "The lazy dog slept in the sun."
tokens1 = document1.lower().split()
tokens2 = document2.lower().split()
terms = list(set(tokens1 + tokens2))
inverted_index = {}
for term in terms:
    documents = []
    if term in tokens1:
        documents.append("Document 1")
    if term in tokens2:
        documents.append("Document 2")
    inverted_index[term] = documents
for term, documents in inverted_index.items():
    print(term, "->", ", ".join(documents))
```

Practical 2b TF-IDF weighting and cosine similarity.

```
import math
from collections import Counter
def tokenize(text):
    return text.lower().split()
def term_frequency(words):
    count = Counter(words)
    total = len(words)
    tf = {}
    for word in count:
        tf[word] = count[word] / total
    return tf
def inverse_document_frequency(documents):
    N = len(documents)
    all_words = set()
    for doc in documents:
        all_words.update(tokenize(doc))
    idf = {}
    for word in all_words:
        doc_count = 0
        for doc in documents:
            if word in set(tokenize(doc)):
                doc_count += 1
        idf[word] = math.log(N / doc_count)
    return idf
def compute_tfidf(tf, idf):
    tfidf = {}
    for word in tf:
        tfidf[word] = tf[word] * idf.get(word, 0)
    return tfidf
def cosine_similarity(vec1, vec2):
    dot_product = 0
    for word in vec1:
        if word in vec2:
            dot_product += vec1[word] * vec2[word]
    norm1 = math.sqrt(sum(value * value for value in vec1.values()))
    norm2 = math.sqrt(sum(value * value for value in vec2.values()))
    if norm1 == 0 or norm2 == 0:
        return 0
```

PRACTICAL – 2A Boolean retrieval model and process queries.

```
documents = {
    1: "apple banana orange",
    2: "apple banana",
    3: "banana orange",
    4: "apple"
}
inverted_index = {}
for doc_id, text in documents.items():
    words = text.split()
    for word in words:
        if word not in inverted_index:
            inverted_index[word] = set()
        inverted_index[word].add(doc_id)
def boolean_and(words):
    result = inverted_index.get(words[0], set())
    for word in words[1:]:
        result = result & inverted_index.get(word, set())
    return sorted(list(result))
def boolean_or(words):
    result = set()
    for word in words:
        result = result | inverted_index.get(word, set())
    return sorted(list(result))
def boolean_not(word, total_docs):
    all_docs = set(range(1, total_docs + 1))
    return sorted(list(all_docs - inverted_index.get(word, set())))

print("Documents containing 'apple' and 'banana':",
      boolean_and(["apple", "banana"]))
print("Documents containing 'apple' or 'orange':",
      boolean_or(["apple", "orange"]))
print("Documents not containing 'orange':", boolean_not("orange",
len(documents)))
```

```
    return dot_product / (norm1 * norm2)
def vsm(documents, query):
    idf = inverse_document_frequency(documents)
    query_tf = term_frequency(tokenize(query))
    query_tfidf = compute_tfidf(query_tf, idf)
    scores = []

    for doc in documents:
        doc_tf = term_frequency(tokenize(doc))
        doc_tfidf = compute_tfidf(doc_tf, idf)
        score = cosine_similarity(query_tfidf, doc_tfidf)
        scores.append(score)
    return scores
docs = [
    "This is the first document.",
    "This document is the second document.",
    "And this is the third one.",
    "Is this the first document?",
]
query = "This is a query document."
print(vsm(docs, query))
```

Practical 3: spelling correction module

```
import difflib
def correct_spelling(query, dictionary):
    query = query.lower()
    return difflib.get_close_matches(query, dictionary, n=5,
cutoff=0.5)
dictionary = ["information", "retrieval", "spelling", "correction",
"system"]
query = input("Enter your query: ")
suggestions = correct_spelling(query, dictionary)
if suggestions:
    print("Did you mean:")
    for word in suggestions:
        print(word)
else:
    print("No suggestions found.")
```

Practical 4 A : Calculate precision, recall, and F-measure for a given set of retrieval results.

```
def calculate_metrics(retrieved_set, relevant_set):
    true_positive = len(retrieved_set & relevant_set)
    false_positive = len(retrieved_set - relevant_set)
    false_negative = len(relevant_set - retrieved_set)
    print("True Positive:", true_positive)
    print("False Positive:", false_positive)
    print("False Negative:", false_negative)
    print()
    precision = true_positive / (true_positive + false_positive) if
(true_positive + false_positive) else 0
    recall = true_positive / (true_positive + false_negative) if
(true_positive + false_negative) else 0
    f_measure = 2 * precision * recall / (precision + recall) if (precision +
recall) else 0
    return precision, recall, f_measure
retrieved_set = {"doc1", "doc2", "doc3"}
relevant_set = {"doc1", "doc4"}
precision, recall, f_measure = calculate_metrics(retrieved_set,
relevant_set)
print("Precision:", precision)
print("Recall:", recall)
print("F-measure:", f_measure)
```

Practical 4 B : Use an evaluation toolkit to measure average precision and other evaluation metrics.

```
from sklearn.metrics import (
    average_precision_score,
    precision_recall_curve,
    auc,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)
import matplotlib.pyplot as plt
import numpy as np
def calculate_classification_metrics(y_true, y_pred_prob):
    y_pred_binary = (np.array(y_pred_prob) >=
0.5).astype(int)
    accuracy = accuracy_score(y_true, y_pred_binary)
    precision = precision_score(y_true, y_pred_binary)
    recall = recall_score(y_true, y_pred_binary)
    f1 = f1_score(y_true, y_pred_binary)
    roc_auc = roc_auc_score(y_true, y_pred_prob)
    average_precision = average_precision_score(y_true,
y_pred_prob)
    precision_curve, recall_curve, _ =
precision_recall_curve(y_true, y_pred_prob)
    area_under_pr_curve = auc(recall_curve,
precision_curve)
    print("Accuracy:", accuracy)
    print("Precision:", precision)
    print("Recall:", recall)
    print("F1 Score:", f1)
    print("ROC-AUC:", roc_auc)
    print("Average Precision:", average_precision)
    print("Area under Precision-Recall curve:",
area_under_pr_curve)
    plt.figure(figsize=(8, 6))
    plt.plot(recall_curve, precision_curve, lw=2,
label="Precision-Recall curve")
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("Precision-Recall Curve")
    plt.legend()
    plt.show()
y_true_binary = [1, 0, 1, 0, 1]
y_pred_prob_binary = [0.8, 0.6, 0.9, 0.5, 0.7]
calculate_classification_metrics(y_true_binary,
y_pred_prob_binary)y_pred_prob_binary = [0.8, 0.6, 0.9,
0.5, 0.7]
calculate_classification_metrics(y_true_binary,
y_pred_prob_binary)
```

PRACTICAL – 05: Implement a text classification algorithm

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

newsgroups = fetch_20newsgroups(
    subset="all",
    remove=("headers", "footers", "quotes")
)
X_train, X_test, y_train, y_test = train_test_split(
    newsgroups.data,
    newsgroups.target,
    test_size=0.2,
    random_state=42
)
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
svm_classifier = SVC(kernel="linear", C=1.0)
svm_classifier.fit(X_train_tfidf, y_train)
y_pred = svm_classifier.predict(X_test_tfidf)
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(
    y_test, y_pred,
    target_names=newsgroups.target_names
)
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n")
print(classification_rep)
```

PRACTICAL – 06: Implement a clustering algorithm

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

docs = [
    "This is the first document.",
    "This document is the second document.",
    "And this is the third one.",
    "Is this the first document?",
    "The last document is here."
]
X = TfidfVectorizer().fit_transform(docs)
kmeans = KMeans(n_clusters=2, n_init=10, random_state=42)
labels = kmeans.fit_predict(X)
score = silhouette_score(X, labels)

print("Cluster Labels:", labels)
print("Silhouette Score:", score)
```

Practical 7: Web Crawling and Indexing

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin

url = "https://www.w3schools.com/python/"
visited = set()

for _ in range(10):
    if url in visited:
        break

    print("Crawled:", url)
    visited.add(url)

    try:
        r = requests.get(url)
        soup = BeautifulSoup(r.text, "html.parser")
        links = soup.find_all("a", href=True)

        if links:
            url = urljoin(url, links[0]["href"])
        else:
            break
    except:
        break
```

PRACTICAL – 08 : Link Analysis and PageRank

```
import numpy as np
```

```
def pagerank(G, d=0.85, max_iter=100):  
    n = len(G)  
  
    M = G / G.sum(axis=1, keepdims=True)  
  
    rank = np.ones(n) / n # initial rank  
  
    for _ in range(max_iter):  
        rank = (1 - d)/n + d * M.T @ rank  
  
    return rank
```

```
G = np.array([  
    [0,1,1,1],  
    [1,0,0,0],  
    [1,1,0,1],  
    [0,0,1,0]  
])
```

```
ranks = pagerank(G)  
for i, score in enumerate(ranks, 1):  
    print(f"Page {i}: {score:.4f}")
```

PRACTICAL – 09: Learning to Rank

```
from sklearn.datasets import make_classification  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC  
from sklearn.metrics import ndcg_score
```

```
X, y = make_classification(n_samples=1000, n_features=10,  
                           random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
                                                    test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

```
model = SVC(kernel="linear")  
model.fit(X_train, y_train)
```

```
scores = model.decision_function(X_test)
```

```
ndcg = ndcg_score([y_test], [scores])  
print("NDCG Score:", ndcg)
```